ReturnFlow: a simpler path to a return

Team: Zejun Xu, Peixuan Ji, Shuchen Liu **Midpoint check-in**

Repository: https://github.com/Zejun-Derry-Xu/DML_Project

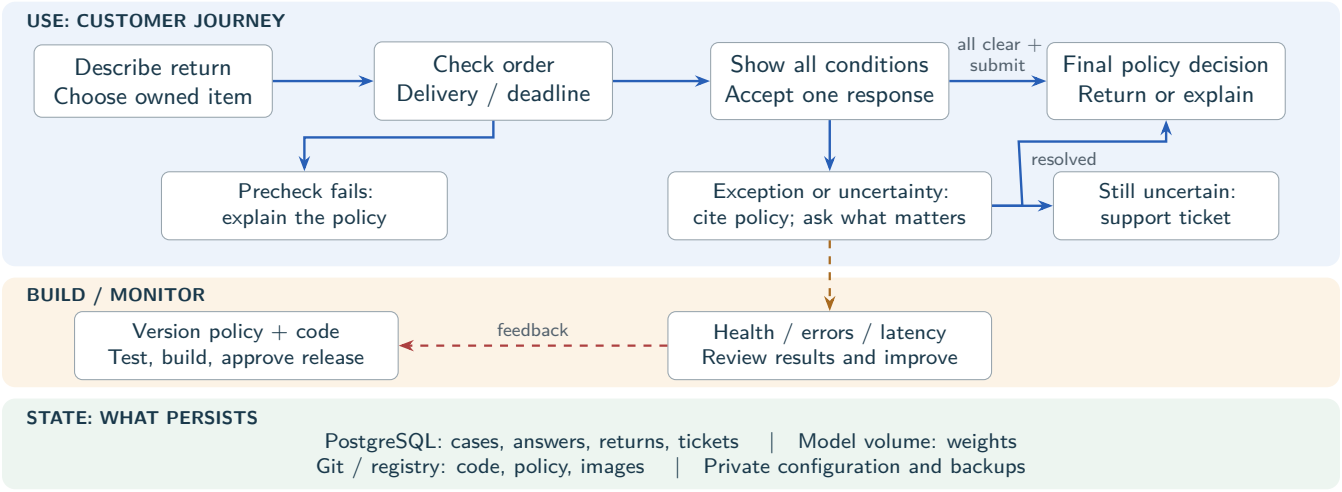
Jetstream2 application entry (last verified October 24):

https://http-149-165-174-228-5173.proxy-js2-iu.exosphere.app/

Deployment status. The URL above served the earlier guided prototype. The latest local candidate adds the checklist-led conversation below; its selected model is Qwen3 4B. Deployment and real-model cloud acceptance for this candidate are pending. The VM is currently shelved; recheck the IP-dependent URL before submission.

1. Solution architecture: what problem do we solve?

Customers should not need to discover every return condition or complete a long series of cards. ReturnFlow accepts a natural-language request, checks trusted order data, then presents the applicable conditions **together**. Customers can confirm them collectively or describe only exceptions. Cards remain an explicit fallback, not the primary goal. The prototype creates simulated return requests, not real refunds.



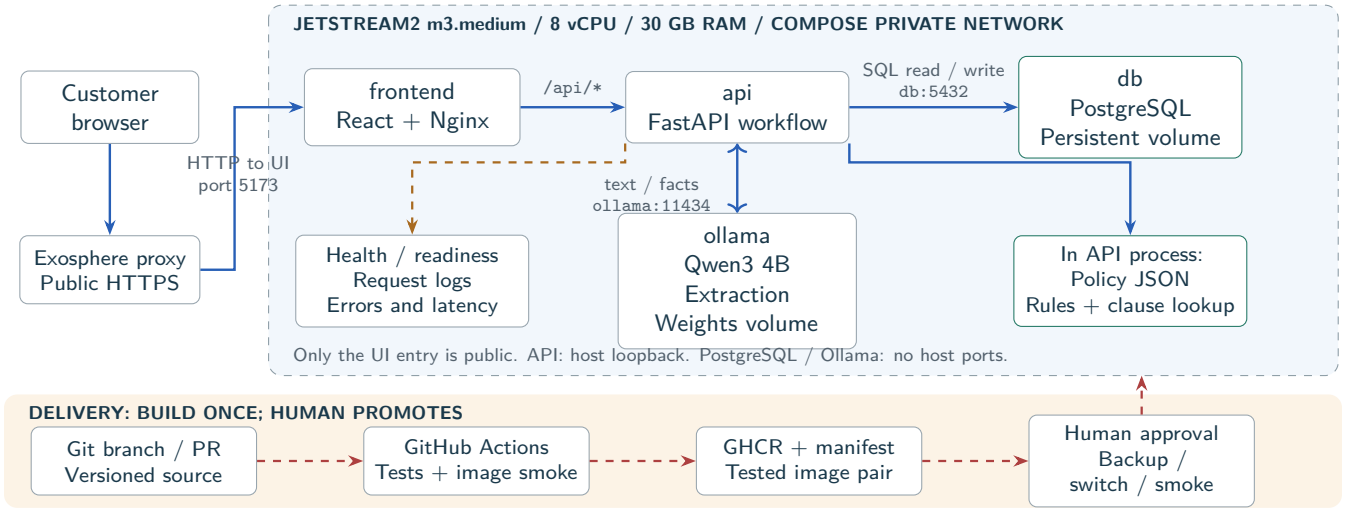
Solid blue: user flow Dashed amber: measurement Dashed red: improvement / release

Brief example in tabular form

Dimension	Our design decisions
Process	**Build:** version policy and test cases; validate images. **Use:** describe, select, review one checklist, confirm or explain exceptions. **Monitor:** inspect health, request failures, latency and outcomes.
Execution	**Frontend:** chat, order cards and a visible checklist. **Backend:** account/order checks, one bounded extraction call, deterministic policy decision and cited clause lookup. The model cannot authorize a return.
State	**PostgreSQL:** cases, checklist snapshots, consent, simulated RMAs and tickets. **Volumes:** database and model weights. **Git / GHCR:** code, policy and image versions. Secrets and backups stay outside Git.

2. Technical architecture: how the components connect

One CPU-only Jetstream2 VM runs Docker Compose. The backend is a **modular monolith**, not a collection of agents: a single FastAPI application owns workflow, policy and transactions. The selected model-enabled topology has **four containers**; guided fallback uses three. This figure specifies the candidate deployment, not a claim that all four services have passed current cloud acceptance.



One request, end to end

1. The browser posts the message and active question ID. Nginx forwards it to FastAPI over the private Compose network; the response returns on the same path.
2. FastAPI checks the fictional account's order ownership and trusted order facts. If configured, one Ollama call interprets text; exact-evidence/type checks filter candidates.
3. Application code binds a collective declaration to the current item, question and policy snapshot, checks exceptions and eligibility, and persists the result in PostgreSQL. A final check and atomic creation prevent duplicate simulated returns.
4. Policy lookup retrieves clauses from the same versioned policy source and returns citations. It explains the policy; it does not replace the eligibility engine.

Representative documented API	Purpose
`POST /api/v1/chat`	Text in; case, explanation and next question out.
`POST /api/v1/questions/{id}/answer`	Explicit card answer or declaration + submission.
`POST /api/v1/policy/ask`	Cited policy help without changing the active return.
`GET /health`, `GET /ready`	Process health; database/schema and version readiness.

Deployment choices. Private environment settings; matching API/frontend image digests; named volumes; restart and health policies. Ollama is CPU-only, one request/model at a time, with 4096 context, 512 output tokens and a 60-second request budget. Qdrant is not required by this lexical/extractive policy path.

3. Prototype, verification and delivery status

A brief user example: normal path and exception

These are intended interactions of the local candidate, not a recorded cloud trial. Assume a fictional customer's owned item has passed delivery, deadline, sale and duplicate-return checks. The reason is optional.

Stage	Customer / interface	System behavior
Request	"The shirt is too loose. I want to return it."	Verify ownership; select the order/item. Do not assume clean, undamaged or intact tags from a fit complaint.
Checklist	Show stains, garment damage, removed tags and damaged tags together. Invite a collective answer or exceptions.	Bind the displayed list to this item, case version, policy and question. Do not preselect an answer.
Normal path	"None of those problems apply, and please submit my return."	In the intended model path, interpret declaration and consent together; perform final checks and create one simulated RMA. The explicit button supports the same action.
Exception path	"The shirt is clean, but the tags were cut off. Can I return it?"	Cite the relevant condition/origin clauses; ask whether this was already true on arrival or happened later. Do not repeat unrelated questions or invent an exception.

CI, smoke and manual continuous delivery

- **CI checks the candidate:** Python lint/types/tests, PostgreSQL migrations and API flows, frontend tests/build, then Linux AMD64 API/frontend image builds. Image smoke runs those exact images with an isolated database and a guided demo flow. Passing this smoke does not establish real Qwen performance.
- **CD prepares a deployable release:** approved triggers publish the tested image pair to GHCR with a commit/digest manifest. A human promotes it; the VM does not rebuild source. This is continuous delivery, not automatic continuous deployment.
- **Deployment protects state:** stop writes, back up PostgreSQL, migrate, switch the paired images, then verify readiness, version identity and policy lookup. Real-model CPU qualification and application smoke are separate gates for LLM mode. Rollback must preserve new records and a compatible schema, not delete volumes.

What is verified, and what remains?

Evidence level	Status as of this draft
Earlier cloud prototype	Public HTTPS interaction verified October 24 for the guided release, not the latest checklist/4B candidate. VM access is not live while shelved.
Latest local candidate	Checklist, collective declaration, exception handling and fallback implemented. Last local backend run: 462 tests, lint and types passed. This is not a new GitHub CI run or real-model cloud qualification.
Model and evaluation	Structured output and evidence checks exist, but recent 4B diagnostics still show unsupported facts. Benchmark work is paused; no formal quality approval is claimed. Planned comparison: frozen rules vs. model-assisted extraction, using precision/recall and request latency, not inferred dialogue-turn savings.
Before submission	Publish a tested candidate pair, explicitly select 4B (the current deployment template defaults to 8B), satisfy the LLM deployment gate, verify public HTTPS, persistence and fallback, then replace this draft's pending status.

Practical limits and next steps

One VM has no failover or zero-downtime upgrades. A monitoring dashboard, off-host backup retention and a two-release rollback rehearsal remain open. Only synthetic accounts and orders are supported; there are no real refunds or physical inspections. Recheck public HTTPS after unshelving. Candidate source: `e835d76` plus local changes, not yet a published release.